

Foundations for AI

Dr. Rishikesh Yadav & Vedant Vibhor

May 2026

School of Mathematical and Statistical Sciences
Indian Institute of Technology Mandi

Table of Contents

1. Introduction
2. Numpy: Foundation of Numerical Computing
3. Pandas: Working with Tabular Data
4. matplotlib and seaborn: Data Visualization & Preprocessing
5. Supervised Learning Fundamentals
6. Model Evaluation
7. TensorFlow

Introduction

Main Objectives

- Build solid foundations with **NumPy**: **arrays**, **vectorization**, and **linear algebra** for AI computations.

Main Objectives

- Build solid foundations with **NumPy**: **arrays**, **vectorization**, and **linear algebra** for AI computations.
- Use **Pandas** to turn raw data into clean tables: **loading**, **cleaning**, and **feature selection**.

Main Objectives

- Build solid foundations with **NumPy**: **arrays**, **vectorization**, and **linear algebra** for AI computations.
- Use **Pandas** to turn raw data into clean tables: **loading**, **cleaning**, and **feature selection**.
- Practice **EDA** to spot **patterns**, **correlations**, and **outliers** before modeling.

Main Objectives

- Build solid foundations with **NumPy**: **arrays**, **vectorization**, and **linear algebra** for AI computations.
- Use **Pandas** to turn raw data into clean tables: **loading**, **cleaning**, and **feature selection**.
- Practice **EDA** to spot **patterns**, **correlations**, and **outliers** before modeling.
- Understand **supervised learning**: mapping X (features) and y (targets) to **regression** and **classification**.

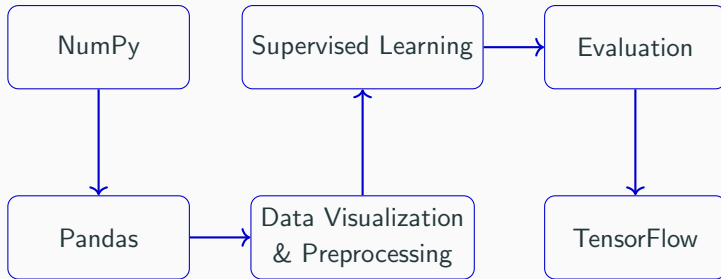
Main Objectives

- Build solid foundations with **NumPy**: **arrays**, **vectorization**, and **linear algebra** for AI computations.
- Use **Pandas** to turn raw data into clean tables: **loading**, **cleaning**, and **feature selection**.
- Practice **EDA** to spot **patterns**, **correlations**, and **outliers** before modeling.
- Understand **supervised learning**: mapping X (features) and y (targets) to **regression** and **classification**.
- Evaluate models using appropriate **metrics** and build intuition for **overfitting vs. generalization**.

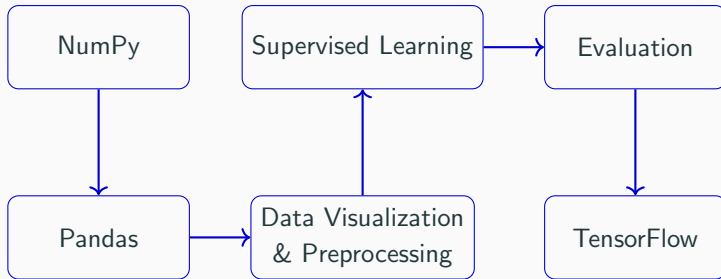
Main Objectives

- Build solid foundations with **NumPy**: **arrays**, **vectorization**, and **linear algebra** for AI computations.
- Use **Pandas** to turn raw data into clean tables: **loading**, **cleaning**, and **feature selection**.
- Practice **EDA** to spot **patterns**, **correlations**, and **outliers** before modeling.
- Understand **supervised learning**: mapping X (features) and y (targets) to **regression** and **classification**.
- Evaluate models using appropriate **metrics** and build intuition for **overfitting vs. generalization**.
- Get started with **TensorFlow/Keras** to define, train, and run a simple **neural network**.

Roadmap

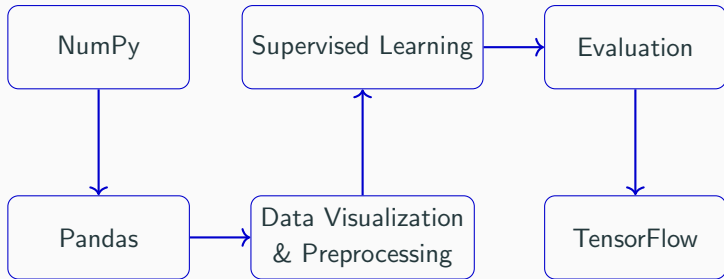


Roadmap



- **NumPy** → fast numerical ops (arrays, broadcasting)
- **Pandas** → real-world data handling (cleaning, missing values)
- **EDA** → plots & correlations to guide choices
- **Supervised learning** → baseline models (regression/classification)
- **Evaluation** → pick the right metric (MAE/RMSE, accuracy/F1)
- **TensorFlow** → train a small neural network

Roadmap



- **NumPy** → fast numerical ops (arrays, broadcasting)
- **Pandas** → real-world data handling (cleaning, missing values)
- **EDA** → plots & correlations to guide choices
- **Supervised learning** → baseline models (regression/classification)
- **Evaluation** → pick the right metric (MAE/RMSE, accuracy/F1)
- **TensorFlow** → train a small neural network

Theory (~2.5–3 hrs) + Hands-on Lab (Separate Session)

Numpy: Foundation of Numerical Computing

Why NumPy for AI?

- **Common array language** across **Pandas**, **scikit-learn**, **PyTorch**, and **TensorFlow**.
- **Speed matters**: vectorized ops run in optimized **C/Fortran** (often via **BLAS/LAPACK**) instead of slow Python loops.
- **Practical toolkit**: broadcasting, random sampling, and linear algebra that show up everywhere in ML/DL.

Why NumPy for AI?

- **Common array language** across **Pandas**, **scikit-learn**, **PyTorch**, and **TensorFlow**.
- **Speed matters**: vectorized ops run in optimized **C/Fortran** (often via **BLAS/LAPACK**) instead of slow Python loops.
- **Practical toolkit**: broadcasting, random sampling, and linear algebra that show up everywhere in ML/DL.

A 30-second history

Python first had multiple array libraries (**Numeric** and **numarray**). In the early 2000s they were unified into **NumPy** (led by **Travis Oliphant**), giving Python a fast, MATLAB-like **n-D array** foundation that today's ML stack builds on.

Why NumPy for AI?

- **Common array language** across **Pandas**, **scikit-learn**, **PyTorch**, and **TensorFlow**.
- **Speed matters**: vectorized ops run in optimized **C/Fortran** (often via **BLAS/LAPACK**) instead of slow Python loops.
- **Practical toolkit**: broadcasting, random sampling, and linear algebra that show up everywhere in ML/DL.

A 30-second history

Python first had multiple array libraries (**Numeric** and **numarray**). In the early 2000s they were unified into **NumPy** (led by **Travis Oliphant**), giving Python a fast, MATLAB-like **n-D array** foundation that today's ML stack builds on.

Core idea: Modern AI = **Data** + **Matrix Operations** + **Optimization**

Python Lists vs NumPy Arrays

Key idea: Python lists are **general containers**; NumPy arrays are **numerical vectors/matrices** designed for fast math.

Python Lists vs NumPy Arrays

Key idea: Python lists are **general containers**; NumPy arrays are **numerical vectors/matrices** designed for fast math.

Python List (container semantics)

```
a = [1, 2, 3]
b = [4, 5, 6]
a + b          # Concatenation
```

Output:

```
[1, 2, 3, 4, 5, 6]
```

NumPy Array (numeric semantics)

```
import numpy as np
a = np.array([1,2,3])
b = np.array([4,5,6])
a + b          # Element-wise addition
```

Output:

```
[5 7 9]
```

Python Lists vs NumPy Arrays

Key idea: Python lists are **general containers**; NumPy arrays are **numerical vectors/matrices** designed for fast math.

Python List (container semantics)

```
a = [1, 2, 3]
b = [4, 5, 6]
a + b          # Concatenation
```

Output:

[1, 2, 3, 4, 5, 6]

Why AI cares: arrays enable **vectorization** (fast C/BLAS under the hood) and make math look like the equations.

NumPy Array (numeric semantics)

```
import numpy as np
a = np.array([1,2,3])
b = np.array([4,5,6])
a + b          # Element-wise addition
```

Output:

[5 7 9]

Understanding Arrays

```
import numpy as np                                # use NumPy

a = np.array([1, 2, 3])                          # create from a Python list
zeros = np.zeros((2, 3))                         # initialize with zeros (shape 2x3)
ones = np.ones((2, 2))                          # initialize with ones (shape 2x2)
eye = np.eye(3)                                  # identity matrix (3x3)
seq = np.arange(0, 10, 2)                        # integer sequence: start, stop, step
lin = np.linspace(0, 1, 5)                       # evenly spaced values between 0 and 1
rand = np.random.rand(2, 3)                     # random samples in [0,1), shape 2x3
```

Understanding Arrays

```
import numpy as np                                # use NumPy

a = np.array([1, 2, 3])                          # create from a Python list
zeros = np.zeros((2, 3))                        # initialize with zeros (shape 2x3)
ones = np.ones((2, 2))                         # initialize with ones (shape 2x2)
eye = np.eye(3)                                 # identity matrix (3x3)
seq = np.arange(0, 10, 2)                      # integer sequence: start, stop, step
lin = np.linspace(0, 1, 5)                    # evenly spaced values between 0 and 1
rand = np.random.rand(2, 3)                   # random samples in [0,1), shape 2x3
```

Outputs (examples)

```
a      → [1 2 3]
zeros → [[0. 0. 0.]
        [0. 0. 0.]]
eye    → [[1. 0. 0.]
        [0. 1. 0.]
        [0. 0. 1.]]
```

Understanding Arrays

```
import numpy as np                                # use NumPy

a = np.array([1, 2, 3])                          # create from a Python list
zeros = np.zeros((2, 3))                         # initialize with zeros (shape 2x3)
ones = np.ones((2, 2))                          # initialize with ones (shape 2x2)
eye = np.eye(3)                                  # identity matrix (3x3)
seq = np.arange(0, 10, 2)                        # integer sequence: start, stop, step
lin = np.linspace(0, 1, 5)                      # evenly spaced values between 0 and 1
rand = np.random.rand(2, 3)                     # random samples in [0,1), shape 2x3
```

Outputs (examples)

```
a      → [1 2 3]
zeros → [[0. 0. 0.]
        [0. 0. 0.]]
eye    → [[1. 0. 0.]
        [0. 1. 0.]
        [0. 0. 1.]]
```

Common AI usage

- **Initialize** weights/biases
- **Create** synthetic batches
- Build **feature matrices** X

Understanding Array Dimensions

```
x = np.array([1,2,3])  
A = np.array([[1,2], [3,4]])
```

Understanding Array Dimensions

```
x = np.array([1,2,3])  
A = np.array([[1,2], [3,4]])
```

Property	x	A
shape	(3,)	(2,2)
ndim	1	2
size	3	4
dtype	int	int

Understanding Array Dimensions

```
x = np.array([1,2,3])  
A = np.array([[1,2], [3,4]])
```

Property	x	A
shape	(3,)	(2,2)
ndim	1	2
size	3	4
dtype	int	int

Terminology

- Scalar: single number
- Vector: 1D array
- Matrix: 2D array
- Tensor: n-dimensional array

Indexing and Slicing

```
a = np.array([10,20,30,40,50])  
print(a[0], a[-1], a[1:4])  
  
A = np.array([[1,2,3], [4,5,6]])  
print(A[0,1], A[:,1], A[1,:])
```

Indexing and Slicing

```
a = np.array([10,20,30,40,50])  
print(a[0], a[-1], a[1:4])  
  
A = np.array([[1,2,3], [4,5,6]])  
print(A[0,1], A[:,1], A[1,:])
```

Output:

```
10 50 [20 30 40]  
2 [2 5] [4 5 6]
```

Indexing and Slicing

```
a = np.array([10,20,30,40,50])  
print(a[0], a[-1], a[1:4])  
  
A = np.array([[1,2,3], [4,5,6]])  
print(A[0,1], A[:,1], A[1,:])
```

Output:

10 50 [20 30 40]

2 [2 5] [4 5 6]

AI Relevance

- Selecting features
- Train-test splitting
- Batch extraction

Vectorized Operations

```
x = np.array([1, 2, 3, 4])  
  
print(x + 10, x * 2, x ** 2)  
print(np.sqrt(x), np.exp(x))
```

Vectorized Operations

```
x = np.array([1, 2, 3, 4])  
  
print(x + 10, x * 2, x ** 2)  
print(np.sqrt(x), np.exp(x))
```

Output:

```
[11 12 13 14] [2 4 6 8] [ 1  4  9 16]  
[1.    1.414 1.732 2.    ] [ 2.718  7.389 20.085 54.598]
```

Vectorized Operations

```
x = np.array([1, 2, 3, 4])  
  
print(x + 10, x * 2, x ** 2)  
print(np.sqrt(x), np.exp(x))
```

Output:

```
[11 12 13 14] [2 4 6 8] [ 1  4  9 16]  
[1.    1.414 1.732 2.    ] [ 2.718  7.389 20.085 54.598]
```

Vectorization

- Apply operations to entire arrays.
- Avoid explicit Python loops.
- Essential for efficient AI computations.

Broadcasting

```
A = np.array([[1,2,3], [4,5,6]])  
b = np.array([10,20,30])  
  
print(A + b)
```


Broadcasting

```
A = np.array([[1,2,3], [4,5,6]])  
b = np.array([10,20,30])  
  
print(A + b)
```

Output:

$$\begin{bmatrix} 11 & 22 & 33 \\ 14 & 25 & 36 \end{bmatrix}$$

Broadcasting

```
A = np.array([[1,2,3], [4,5,6]])  
b = np.array([10,20,30])  
  
print(A + b)
```

Output:

$$\begin{bmatrix} 11 & 22 & 33 \\ 14 & 25 & 36 \end{bmatrix}$$

Broadcasting Rule

- Smaller arrays automatically expand.
- Eliminates unnecessary copying.

AI Example: Adding bias terms in neural networks.

Reshaping Arrays

```
x = np.arange(12)
print(x.reshape(3, 4))
print(x.reshape(2, 6))
print(x.flatten())
```

Reshaping Arrays

```
x = np.arange(12)
print(x.reshape(3, 4))
print(x.reshape(2, 6))
print(x.flatten())
```

Output (partial):

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

Reshaping Arrays

```
x = np.arange(12)
print(x.reshape(3, 4))
print(x.reshape(2, 6))
print(x.flatten())
```

Output (partial):

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

Why Important?

- Images need reshaping.
- Data preparation for ML models.
- Mini-batch organization.

Aggregation Functions

```
x = np.array([2,4,6,8])  
  
print(np.sum(x))  
print(np.mean(x), np.std(x))  
print(np.min(x), np.max(x))
```

Output:

```
20  
5.0 2.236...  
2 8
```

Used For

- Data analysis
- Feature engineering
- Model evaluation metrics

Working Along Axes

```
A = np.array([[1,2,3],
              [4,5,6]])

print(A.sum(axis=0))    # column-wise
print(A.sum(axis=1))    # row-wise
print(A.mean(axis=0))
```

Output:

[5 7 9]

[6 15]

[2.5 3.5 4.5]

AI Example

- Compute feature means.
- Normalize datasets.

Linear Algebra Essentials

```
A = np.array([[1,2],[3,4]])  
B = np.array([[5,6],[7,8]])  
  
print(A @ B)           # Matrix multiplication  
print(A.T)             # Transpose  
print(np.linalg.det(A))
```

Output:

```
[[19 22]  
 [43 50]]  
[[1 3]  
 [2 4]]  
-2.0
```


Linear Algebra Essentials

```
A = np.array([[1,2],[3,4]])  
B = np.array([[5,6],[7,8]])  
  
print(A @ B)           # Matrix multiplication  
print(A.T)             # Transpose  
print(np.linalg.det(A))
```

Output:

```
[[19 22]  
 [43 50]]  
[[1 3]  
 [2 4]]  
-2.0
```

Really essential for neural networks, PCA, optimization.

NumPy in Action: Linear Model

$$y = w_1x_1 + w_2x_2 + b$$

```
X = np.array([[2,3], [4,5], [6,7]])  
w = np.array([0.5, 1.2])  
b = 2  
  
pred = X @ w + b  
print(pred)
```

Output:

```
[ 6.6  10.0 13.4]
```

NumPy in Action: Linear Model

$$y = w_1x_1 + w_2x_2 + b$$

```
X = np.array([[2,3], [4,5], [6,7]])  
w = np.array([0.5, 1.2])  
b = 2  
  
pred = X @ w + b  
print(pred)
```

Output:

```
[ 6.6  10.0 13.4]
```

This matrix operation is the foundation of neural network forward passes.

Random Numbers for AI

```
np.random.seed(42)

print(np.random.rand(3))
print(np.random.randn(3))
print(np.random.randint(0, 10, 5))
```

Output (example):

```
[0.3745 0.9507 0.7320]
[-1.1119 0.3189 0.2790]
[7 2 5 4 1]
```

Random Numbers for AI

```
np.random.seed(42)

print(np.random.rand(3))
print(np.random.randn(3))
print(np.random.randint(0, 10, 5))
```

Output (example):

```
[0.3745 0.9507 0.7320]
[-1.1119 0.3189 0.2790]
[7 2 5 4 1]
```

Applications: Weight initialization, data augmentation, stochastic optimization.

Mini Exercise (1/2)

Given:

```
X = np.array([[1,2],  
              [3,4],  
              [5,6]])
```

Find:

1. Shape of X

Mini Exercise (1/2)

Given:

```
X = np.array([[1,2],  
              [3,4],  
              [5,6]])
```

Find:

1. Shape of X

```
X.shape
```

(3, 2)

2. Mean of each column

Mini Exercise (1/2)

Given:

```
X = np.array([[1,2],  
              [3,4],  
              [5,6]])
```

Find:

1. Shape of X

```
X.shape
```

(3, 2)

2. Mean of each column

```
X.mean(axis=0)
```

[3. 4.]

Mini Exercise (2/2)

3. Sum of each row

Mini Exercise (2/2)

3. Sum of each row

```
X.sum(axis=1)
```

```
[ 3  7 11]
```

4. X multiplied by 10

Mini Exercise (2/2)

3. Sum of each row

```
X.sum(axis=1)
```

```
[ 3  7 11]
```

4. X multiplied by 10

```
X * 10
```

```
[[10 20]
```

```
 [30 40]
```

```
 [50 60]]
```

5. X transpose

Mini Exercise (2/2)

3. Sum of each row

```
X.sum(axis=1)
```

```
[ 3  7 11]
```

4. X multiplied by 10

```
X * 10
```

```
[[10 20]
```

```
 [30 40]
```

```
 [50 60]]
```

5. X transpose

```
X.T
```

```
[[1 3 5]
```

```
 [2 4 6]]
```

Key Takeaways

- **ndarray** is NumPy's main object: a fast, contiguous **n-dimensional array** used to store vectors, matrices, and tensors.

Key Takeaways

- **ndarray** is NumPy's main object: a fast, contiguous **n-dimensional array** used to store vectors, matrices, and tensors.
- Always check **shape** and **ndim** before operations; most bugs in ML code are **dimension/axis** mistakes.

Key Takeaways

- **ndarray** is NumPy's main object: a fast, contiguous **n-dimensional array** used to store vectors, matrices, and tensors.
- Always check **shape** and **ndim** before operations; most bugs in ML code are **dimension/axis** mistakes.
- **Vectorization** (operate on whole arrays) replaces Python loops and is the reason NumPy code is **fast**.

Key Takeaways

- **ndarray** is NumPy's main object: a fast, contiguous **n-dimensional array** used to store vectors, matrices, and tensors.
- Always check **shape** and **ndim** before operations; most bugs in ML code are **dimension/axis** mistakes.
- **Vectorization** (operate on whole arrays) replaces Python loops and is the reason NumPy code is **fast**.
- **Broadcasting** lets different shapes work together (e.g., add a bias vector to every row of a batch) without manual copying.

Key Takeaways

- **ndarray** is NumPy's main object: a fast, contiguous **n-dimensional array** used to store vectors, matrices, and tensors.
- Always check **shape** and **ndim** before operations; most bugs in ML code are **dimension/axis** mistakes.
- **Vectorization** (operate on whole arrays) replaces Python loops and is the reason NumPy code is **fast**.
- **Broadcasting** lets different shapes work together (e.g., add a bias vector to every row of a batch) without manual copying.
- Learn the "daily tools": **indexing/slicing**, **reshape/flatten**, **axis** reductions (sum/mean), and **matrix multiplication** (@).

Key Takeaways

- **ndarray** is NumPy's main object: a fast, contiguous **n-dimensional array** used to store vectors, matrices, and tensors.
- Always check **shape** and **ndim** before operations; most bugs in ML code are **dimension/axis** mistakes.
- **Vectorization** (operate on whole arrays) replaces Python loops and is the reason NumPy code is **fast**.
- **Broadcasting** lets different shapes work together (e.g., add a bias vector to every row of a batch) without manual copying.
- Learn the "daily tools": **indexing/slicing**, **reshape/flatten**, **axis** reductions (sum/mean), and **matrix multiplication** (@).
- Strong NumPy fundamentals make **Pandas**, **Scikit-Learn**, and **TensorFlow/PyTorch** much easier.

Key Takeaways

- **ndarray** is NumPy's main object: a fast, contiguous **n-dimensional array** used to store vectors, matrices, and tensors.
- Always check **shape** and **ndim** before operations; most bugs in ML code are **dimension/axis** mistakes.
- **Vectorization** (operate on whole arrays) replaces Python loops and is the reason NumPy code is **fast**.
- **Broadcasting** lets different shapes work together (e.g., add a bias vector to every row of a batch) without manual copying.
- Learn the "daily tools": **indexing/slicing**, **reshape/flatten**, **axis** reductions (sum/mean), and **matrix multiplication** (@).
- Strong NumPy fundamentals make **Pandas**, **Scikit-Learn**, and **TensorFlow/PyTorch** much easier.

Key Takeaways

- **ndarray** is NumPy's main object: a fast, contiguous **n-dimensional array** used to store vectors, matrices, and tensors.
- Always check **shape** and **ndim** before operations; most bugs in ML code are **dimension/axis** mistakes.
- **Vectorization** (operate on whole arrays) replaces Python loops and is the reason NumPy code is **fast**.
- **Broadcasting** lets different shapes work together (e.g., add a bias vector to every row of a batch) without manual copying.
- Learn the "daily tools": **indexing/slicing**, **reshape/flatten**, **axis** reductions (sum/mean), and **matrix multiplication** (@).
- Strong NumPy fundamentals make **Pandas**, **Scikit-Learn**, and **TensorFlow/PyTorch** much easier.

Next Step: NumPy → Pandas → Scikit-Learn → Deep Learning

Pandas: Working with Tabular Data

Why Pandas?

Pandas is Python's standard library for working with **tabular data** (rows & columns).

Why Pandas?

Pandas is Python's standard library for working with **tabular data** (rows & columns).

- Built on top of **NumPy** $\Rightarrow$ fast operations on large datasets.

Why Pandas?

Pandas is Python's standard library for working with **tabular data** (rows & columns).

- Built on top of **NumPy** $\Rightarrow$ fast operations on large datasets.
- The **DataFrame** is like an Excel table with **column names**, **row indices**, and mixed data types.

Why Pandas?

Pandas is Python's standard library for working with **tabular data** (rows & columns).

- Built on top of **NumPy** $\Rightarrow$ fast operations on large datasets.
- The **DataFrame** is like an Excel table with **column names**, **row indices**, and mixed data types.
- Makes **data cleaning** easy: handle missing values, fix types, remove duplicates.

Why Pandas?

Pandas is Python's standard library for working with **tabular data** (rows & columns).

- Built on top of **NumPy** $\Rightarrow$ fast operations on large datasets.
- The **DataFrame** is like an Excel table with **column names**, **row indices**, and mixed data types.
- Makes **data cleaning** easy: handle missing values, fix types, remove duplicates.
- Supports **filtering**, **groupby/aggregation**, and **feature engineering** in a few lines.

Why Pandas?

Pandas is Python's standard library for working with **tabular data** (rows & columns).

- Built on top of **NumPy** $\Rightarrow$ fast operations on large datasets.
- The **DataFrame** is like an Excel table with **column names**, **row indices**, and mixed data types.
- Makes **data cleaning** easy: handle missing values, fix types, remove duplicates.
- Supports **filtering**, **groupby/aggregation**, and **feature engineering** in a few lines.
- Bridges to ML tools: select features, then export to **NumPy arrays** for models.

Why Pandas?

Pandas is Python's standard library for working with **tabular data** (rows & columns).

- Built on top of **NumPy** $\Rightarrow$ fast operations on large datasets.
- The **DataFrame** is like an Excel table with **column names**, **row indices**, and mixed data types.
- Makes **data cleaning** easy: handle missing values, fix types, remove duplicates.
- Supports **filtering**, **groupby/aggregation**, and **feature engineering** in a few lines.
- Bridges to ML tools: select features, then export to **NumPy arrays** for models.

Why Pandas?

Pandas is Python's standard library for working with **tabular data** (rows & columns).

- Built on top of **NumPy** $\Rightarrow$ fast operations on large datasets.
- The **DataFrame** is like an Excel table with **column names**, **row indices**, and mixed data types.
- Makes **data cleaning** easy: handle missing values, fix types, remove duplicates.
- Supports **filtering**, **groupby/aggregation**, and **feature engineering** in a few lines.
- Bridges to ML tools: select features, then export to **NumPy arrays** for models.

Typical workflow: Raw data $\rightarrow$ Pandas $\rightarrow$ Clean & explore $\rightarrow$ Features $\rightarrow$ Modeling

Core Data Structures

- **Series:** 1-dimensional labeled array
- **DataFrame:** 2-dimensional table with rows and columns (most used)

```
import pandas as pd

s = pd.Series([10, 20, 30, 40])
print(s)
```

Core Data Structures

- **Series:** 1-dimensional labeled array
- **DataFrame:** 2-dimensional table with rows and columns (most used)

```
import pandas as pd

s = pd.Series([10, 20, 30, 40])
print(s)
```

Output:

0 10

1 20

2 30

3 40

dtype: int64

Creating a DataFrame

```
import pandas as pd

data = {
    "Name": ["Alice", "Bob", "Charlie"],
    "Age": [22, 24, 23],
    "Score": [90, 85, 95]
}

df = pd.DataFrame(data)
print(df)
```


Creating a DataFrame

```
import pandas as pd

data = {
    "Name": ["Alice", "Bob", "Charlie"],
    "Age": [22, 24, 23],
    "Score": [90, 85, 95]
}

df = pd.DataFrame(data)
print(df)
```

Output:

	Name	Age	Score
0	Alice	22	90
1	Bob	24	85
2	Charlie	23	95

Reading Data Files

```
df = pd.read_csv("students.csv") # df = pd.read_excel("students.xlsx")  
print(df.head())  
print(df.shape)  
print(df.info())
```

Reading Data Files

```
df = pd.read_csv("students.csv") # df = pd.read_excel("students.xlsx")
print(df.head())
print(df.shape)
print(df.info())
```

Common Output:

	Hours	Attendance	Marks
0	2	70	55
1	4	75	60
...			

(100, 3)

<class 'pandas.core.frame.DataFrame'>

...

Reading Data Files

```
df = pd.read_csv("students.csv") # df = pd.read_excel("students.xlsx")
print(df.head())
print(df.shape)
print(df.info())
```

Common Output:

	Hours	Attendance	Marks
0	2	70	55
1	4	75	60
...			

(100, 3)

<class 'pandas.core.frame.DataFrame'>

...

Note: Most machine learning projects start with CSV files.

Selecting Columns

```
print(df["Age"])  
print(df[["Age", "Score"]])
```

Selecting Columns

```
print(df["Age"])  
print(df[["Age", "Score"]])
```

Output (partial):

0 22

1 24

2 23

Name: Age, dtype: int64

	Age	Score
0	22	90
1	24	85
2	23	95

Selecting columns = Selecting features for ML models.

Row Selection with loc and iloc

```
print(df.loc[0])           # label-based  
print(df.iloc[0:2])        # position-based
```

Output:

Name Alice

Age 22

Score 90

Name: 0, dtype: object

	Name	Age	Score
0	Alice	22	90
1	Bob	24	85

Key Difference: loc uses labels, iloc uses integer positions.

Filtering Data

```
print(df[df["Age"] > 22])  
  
print(df[(df["Age"] > 22) & (df["Score"] > 85)])
```

Output (example):

	Name	Age	Score
1	Bob	24	85
2	Charlie	23	95

	Name	Age	Score
2	Charlie	23	95

Useful for selecting specific observations before training.

Adding New Columns & Feature Engineering

```
df["Bonus"] = 5  
df["FinalScore"] = df["Score"] + df["Bonus"]  
  
print(df[["Score", "Bonus", "FinalScore"]])
```

Output:

	Score	Bonus	FinalScore
0	90	5	95
1	85	5	90
2	95	5	100

Feature engineering is a key step in improving model performance.

Descriptive Statistics

```
print(df.describe())

print("Mean Age:", df["Age"].mean())
print("Max Score:", df["Score"].max())
```

Output (partial):

	Age	Score
count	3.000000	3.000000
mean	23.000000	90.000000
std	1.000000	5.000000
...		

Essential for understanding your dataset quickly.

Handling Missing Values

```
print(df.isnull().sum())

df_clean = df.dropna()          # Remove
# df.fillna(df.mean(), inplace=True) # Fill
```

Output:

```
Name      0
Age        0
Score      0
dtype: int64
```

Data cleaning is often the most time-consuming part of ML projects.

Combining Pandas with NumPy

```
import pandas as pd
import numpy as np

# Assume df is already a pandas DataFrame
X = df[["Age", "Score"]].values    # Convert to NumPy array
print(type(X))
print(X.shape)
```

Output:

```
<class 'numpy.ndarray'>
(3, 2)
```

This NumPy matrix is what gets fed into machine learning models.

Mini Machine Learning Workflow

```
import pandas as pd
import numpy as np
df = pd.read_csv("students.csv")

X = df[["Hours", "Attendance"]].values
y = df["Marks"].values

print(X.shape, y.shape)
```

Mini Machine Learning Workflow

```
import pandas as pd
import numpy as np
df = pd.read_csv("students.csv")

X = df[["Hours", "Attendance"]].values
y = df["Marks"].values

print(X.shape, y.shape)
```

Pipeline Summary:

1. Load data with Pandas

Mini Machine Learning Workflow

```
import pandas as pd
import numpy as np
df = pd.read_csv("students.csv")

X = df[["Hours", "Attendance"]].values
y = df["Marks"].values

print(X.shape, y.shape)
```

Pipeline Summary:

1. Load data with Pandas
2. Clean & explore

Mini Machine Learning Workflow

```
import pandas as pd
import numpy as np
df = pd.read_csv("students.csv")

X = df[["Hours", "Attendance"]].values
y = df["Marks"].values

print(X.shape, y.shape)
```

Pipeline Summary:

1. Load data with Pandas
2. Clean & explore
3. Select features

Mini Machine Learning Workflow

```
import pandas as pd
import numpy as np
df = pd.read_csv("students.csv")

X = df[["Hours", "Attendance"]].values
y = df["Marks"].values

print(X.shape, y.shape)
```

Pipeline Summary:

1. Load data with Pandas
2. Clean & explore
3. Select features
4. Convert to NumPy

Mini Machine Learning Workflow

```
import pandas as pd
import numpy as np
df = pd.read_csv("students.csv")

X = df[["Hours", "Attendance"]].values
y = df["Marks"].values

print(X.shape, y.shape)
```

Pipeline Summary:

1. Load data with Pandas
2. Clean & explore
3. Select features
4. Convert to NumPy
5. Train model

Pandas Mini Exercise

Given:

```
df = pd.DataFrame({  
    "Name": ["A", "B", "C"],  
    "Age": [20, 25, 22],  
    "Marks": [80, 95, 88]  
})
```

Tasks (write the Pandas code):

1. Compute **mean age**

(hint: `df["Age"].mean()`)

Pandas Mini Exercise

Given:

```
df = pd.DataFrame({  
    "Name": ["A", "B", "C"],  
    "Age": [20, 25, 22],  
    "Marks": [80, 95, 88]  
})
```

Tasks (write the Pandas code):

1. Compute **mean age**
2. **Filter** students with **Marks > 85**

(hint: `df["Age"].mean()`)

(hint: boolean mask)

Pandas Mini Exercise

Given:

```
df = pd.DataFrame({  
    "Name": ["A", "B", "C"],  
    "Age": [20, 25, 22],  
    "Marks": [80, 95, 88]  
})
```

Tasks (write the Pandas code):

1. Compute **mean age**
2. **Filter** students with **Marks** > 85
3. Add a **Bonus** column (+5)

(hint: `df["Age"].mean()`)

(hint: boolean mask)

(hint: assignment)

Pandas Mini Exercise

Given:

```
df = pd.DataFrame({  
    "Name": ["A", "B", "C"],  
    "Age": [20, 25, 22],  
    "Marks": [80, 95, 88]  
})
```

Tasks (write the Pandas code):

1. Compute **mean age**
2. **Filter** students with **Marks** > 85
3. Add a **Bonus** column (+5)
4. **Sort** by Marks (descending)

(hint: `df["Age"].mean()`)

(hint: boolean mask)

(hint: assignment)

(hint: `sort_values`)

Pandas Mini Exercise

Given:

```
df = pd.DataFrame({  
    "Name": ["A", "B", "C"],  
    "Age": [20, 25, 22],  
    "Marks": [80, 95, 88]  
})
```

Tasks (write the Pandas code):

1. Compute **mean age**
2. **Filter** students with **Marks** > 85
3. Add a **Bonus** column (+5)
4. **Sort** by Marks (descending)
5. Convert **Marks** to a **NumPy array**

(hint: `df["Age"].mean()`)

(hint: boolean mask)

(hint: assignment)

(hint: `sort_values`)

(hint: `to_numpy()`)

Pandas Key Takeaways

- **DataFrame** = the central object for **tabular data** (like a spreadsheet, but programmable).

Pandas Key Takeaways

- **DataFrame** = the central object for **tabular data** (like a spreadsheet, but programmable).
- Use **loc/iloc** for reliable row selection and **column selection** for features.

Pandas Key Takeaways

- **DataFrame** = the central object for **tabular data** (like a spreadsheet, but programmable).
- Use **loc/iloc** for reliable row selection and **column selection** for features.
- Most ML prep is **cleaning** + **feature engineering** (missing values, new columns, transforms).

Pandas Key Takeaways

- **DataFrame** = the central object for **tabular data** (like a spreadsheet, but programmable).
- Use **loc/iloc** for reliable row selection and **column selection** for features.
- Most ML prep is **cleaning** + **feature engineering** (missing values, new columns, transforms).
- Before modeling, convert to **NumPy**: `X = df[cols].to_numpy()` and `y = df[target].to_numpy()`.

Pandas Key Takeaways

- **DataFrame** = the central object for **tabular data** (like a spreadsheet, but programmable).
- Use **loc/iloc** for reliable row selection and **column selection** for features.
- Most ML prep is **cleaning** + **feature engineering** (missing values, new columns, transforms).
- Before modeling, convert to **NumPy**: `X = df[cols].to_numpy()` and `y = df[target].to_numpy()`.

Pandas Key Takeaways

- **DataFrame** = the central object for **tabular data** (like a spreadsheet, but programmable).
- Use **loc/iloc** for reliable row selection and **column selection** for features.
- Most ML prep is **cleaning** + **feature engineering** (missing values, new columns, transforms).
- Before modeling, convert to **NumPy**: `X = df[cols].to_numpy()` and `y = df[target].to_numpy()`.

NumPy + **Pandas** = Foundation of AI Data Processing

matplotlib and seaborn: Data Visualization & Preprocessing

Why Visualize First?

Data visualization turns raw numbers into **insights** you can act on.

Visualize → Understand → Clean → Engineer features → Model

Why Visualize First?

Data **visualization** turns raw numbers into **insights** you can act on.

Visualize → Understand → Clean → Engineer features → Model

Key benefits:

- Discover **patterns** and **relationships** (signal).

Why Visualize First?

Data **visualization** turns raw numbers into **insights** you can act on.

Visualize → Understand → Clean → Engineer features → Model

Key benefits:

- Discover **patterns** and **relationships** (signal).
- Spot **outliers** and **data quality** issues early.

Why Visualize First?

Data **visualization** turns raw numbers into **insights** you can act on.

Visualize → Understand → Clean → Engineer features → Model

Key benefits:

- Discover **patterns** and **relationships** (signal).
- Spot **outliers** and **data quality** issues early.
- Guide **feature engineering** decisions.

Why Visualize First?

Data **visualization** turns raw numbers into **insights** you can act on.

Visualize → Understand → Clean → Engineer features → Model

Key benefits:

- Discover **patterns** and **relationships** (signal).
- Spot **outliers** and **data quality** issues early.
- Guide **feature engineering** decisions.
- Build intuition before trying complex models (debug faster).

Visualization in the AI Workflow

Raw data → Cleaning → Visualization → Feature engineering →
Modeling

Visualization helps you answer:

- Are variables **correlated** (useful signal)?

Raw data → Cleaning → Visualization → Feature engineering →
Modeling

Visualization helps you answer:

- Are variables **correlated** (useful signal)?
- Are there **outliers** or **skewed** distributions?

Raw data → Cleaning → Visualization → Feature engineering →
Modeling

Visualization helps you answer:

- Are variables **correlated** (useful signal)?
- Are there **outliers** or **skewed** distributions?
- Which **features** look promising before modeling?

Matplotlib Basics

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

plt.figure(figsize=(5,3))
plt.plot(x, y, marker='o')
plt.xlabel("Hours Studied")
plt.ylabel("Marks")
plt.title("Study Hours vs Performance")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

Matplotlib Basics

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

plt.figure(figsize=(5,3))
plt.plot(x, y, marker='o')
plt.xlabel("Hours Studied")
plt.ylabel("Marks")
plt.title("Study Hours vs Performance")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

What to look for: trend, outliers, and non-linearity.

Seaborn Basics

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")

# Quick plot directly from a DataFrame
sns.scatterplot(data=df, x="Hours", y="Marks", hue="Attendance")
plt.title("Hours vs Marks (colored by Attendance)")
plt.show()
```

Seaborn Basics

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")

# Quick plot directly from a DataFrame
sns.scatterplot(data=df, x="Hours", y="Marks", hue="Attendance")
plt.title("Hours vs Marks (colored by Attendance)")
plt.show()
```

Why Seaborn (vs Matplotlib)?

- **Cleaner defaults:** themes, grids, nicer colors.

Seaborn Basics

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")

# Quick plot directly from a DataFrame
sns.scatterplot(data=df, x="Hours", y="Marks", hue="Attendance")
plt.title("Hours vs Marks (colored by Attendance)")
plt.show()
```

Why Seaborn (vs Matplotlib)?

- **Cleaner defaults:** themes, grids, nicer colors.
- **DataFrame-friendly:** works directly with `data=...`, column names, and grouping (`hue`).

Seaborn Basics

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")

# Quick plot directly from a DataFrame
sns.scatterplot(data=df, x="Hours", y="Marks", hue="Attendance")
plt.title("Hours vs Marks (colored by Attendance)")
plt.show()
```

Why Seaborn (vs Matplotlib)?

- **Cleaner defaults:** themes, grids, nicer colors.
- **DataFrame-friendly:** works directly with `data=...`, column names, and grouping (`hue`).
- **Statistical plots** built-in: KDE, violin plots, regression plots.

Seaborn Basics

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")

# Quick plot directly from a DataFrame
sns.scatterplot(data=df, x="Hours", y="Marks", hue="Attendance")
plt.title("Hours vs Marks (colored by Attendance)")
plt.show()
```

Why Seaborn (vs Matplotlib)?

- **Cleaner defaults**: themes, grids, nicer colors.
- **DataFrame-friendly**: works directly with `data=...`, column names, and grouping (`hue`).
- **Statistical plots** built-in: KDE, violin plots, regression plots.
- Still uses Matplotlib underneath $\Rightarrow$ you can customize everything with `plt`.

Common Plot Types

Plot type	When to use it	Python command
Scatter	Relationship between two numeric variables	<code>sns.scatterplot(...)</code>
Histogram	Shape of a distribution (skew, multi-modal)	<code>plt.hist(...)</code> <code>sns.kdeplot(...)</code>
Box plot	Outliers + spread (median, IQR)	<code>sns.boxplot(...)</code>
Bar plot	Compare categories / counts	<code>sns.barplot(...)</code>
Heatmap	Correlations across many features	<code>sns.heatmap(...)</code>

Note: `sns` is [Seaborn](#) (typically: `import seaborn as sns`).

Common Plot Types

Plot type	When to use it	Python command
Scatter	Relationship between two numeric variables	<code>sns.scatterplot(...)</code>
Histogram	Shape of a distribution (skew, multi-modal)	<code>plt.hist(...)</code> <code>sns.kdeplot(...)</code>
Box plot	Outliers + spread (median, IQR)	<code>sns.boxplot(...)</code>
Bar plot	Compare categories / counts	<code>sns.barplot(...)</code>
Heatmap	Correlations across many features	<code>sns.heatmap(...)</code>

Note: `sns` is [Seaborn](#) (typically: `import seaborn as sns`).

Overall idea: One good plot that answers a question beats ten random plots.

Case Study 1: Scatter Plot (Relationships)

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.scatterplot(data=df, x="Hours", y="Marks")
plt.title("Hours Studied vs Marks")
plt.show()
```


Case Study 1: Scatter Plot (Relationships)

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.scatterplot(data=df, x="Hours", y="Marks")
plt.title("Hours Studied vs Marks")
plt.show()
```

Interpretation:

- Look for **correlation** (upward/downward pattern).
- If the scatter is tight, **Hours** is likely a strong predictive feature.
- If it curves, you may need **non-linear** features or models.

Case Study 2: Histogram (Distributions)

```
import matplotlib.pyplot as plt

plt.hist(df["Marks"], bins=10, edgecolor='black')
plt.title("Distribution of Student Marks")
plt.xlabel("Marks")
plt.ylabel("Frequency")
plt.show()
```

Case Study 2: Histogram (Distributions)

```
import matplotlib.pyplot as plt

plt.hist(df["Marks"], bins=10, edgecolor='black')
plt.title("Distribution of Student Marks")
plt.xlabel("Marks")
plt.ylabel("Frequency")
plt.show()
```

Observations:

- Is it **skewed** (long tail) or roughly **normal**?

Case Study 2: Histogram (Distributions)

```
import matplotlib.pyplot as plt

plt.hist(df["Marks"], bins=10, edgecolor='black')
plt.title("Distribution of Student Marks")
plt.xlabel("Marks")
plt.ylabel("Frequency")
plt.show()
```

Observations:

- Is it **skewed** (long tail) or roughly **normal**?
- Are there **multiple peaks** (different groups in data)?

Case Study 2: Histogram (Distributions)

```
import matplotlib.pyplot as plt

plt.hist(df["Marks"], bins=10, edgecolor='black')
plt.title("Distribution of Student Marks")
plt.xlabel("Marks")
plt.ylabel("Frequency")
plt.show()
```

Observations:

- Is it **skewed** (long tail) or roughly **normal**?
- Are there **multiple peaks** (different groups in data)?
- Distribution shape guides **preprocessing** and model choice.

Case Study 3: Box Plot (Outliers)

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.boxplot(data=df, x="Marks")
plt.title("Marks Distribution (Outliers)")
plt.show()
```

Case Study 3: Box Plot (Outliers)

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.boxplot(data=df, x="Marks")
plt.title("Marks Distribution (Outliers)")
plt.show()
```

Note: Outliers can strongly affect training. Investigate them before removing.

Seaborn & Correlation Heatmap

```
import matplotlib.pyplot as plt
import seaborn as sns

corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Feature Correlation Matrix")
plt.show()
```


Seaborn & Correlation Heatmap

```
import matplotlib.pyplot as plt
import seaborn as sns

corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Feature Correlation Matrix")
plt.show()
```

Key insights:

- Spot **strong positive/negative correlations**.

Seaborn & Correlation Heatmap

```
import matplotlib.pyplot as plt
import seaborn as sns

corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Feature Correlation Matrix")
plt.show()
```

Key insights:

- Spot **strong positive/negative correlations**.
- Detect **redundant features** (very high correlation).

Seaborn & Correlation Heatmap

```
import matplotlib.pyplot as plt
import seaborn as sns

corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Feature Correlation Matrix")
plt.show()
```

Key insights:

- Spot **strong positive/negative correlations**.
- Detect **redundant features** (very high correlation).
- Identify likely **predictors** for the target variable.

Data Preprocessing Essentials

Real-world data is rarely **clean**. **Preprocessing** is crucial.

Data Preprocessing Essentials

Real-world data is rarely **clean**. **Preprocessing** is crucial.

Common tasks:

- Handle **missing values**
- Remove **duplicate records**
- **Scale features** (standardize/normalize)
- **Encode categorical** variables (make them numeric)
- Investigate **outliers** (fix, cap, or remove)

Data Preprocessing Essentials

Real-world data is rarely **clean**. **Preprocessing** is crucial.

Common tasks:

- Handle **missing values**
- Remove **duplicate records**
- **Scale features** (standardize/normalize)
- **Encode categorical** variables (make them numeric)
- Investigate **outliers** (fix, cap, or remove)

Rule of thumb: **60–80%** of ML project time goes into data preparation.

Handling Missing Values

```
print(df.isnull().sum())

# Option 1: Remove rows with missing entries
df_clean = df.dropna()

# Option 2: Fill missing values (numeric columns)
# df.fillna(df.mean(numeric_only=True), inplace=True)
```

Handling Missing Values

```
print(df.isnull().sum())

# Option 1: Remove rows with missing entries
df_clean = df.dropna()

# Option 2: Fill missing values (numeric columns)
# df.fillna(df.mean(numeric_only=True), inplace=True)
```

Output (example):

```
Hours          0
Attendance     2
Marks          0
dtype: int64
```

Tip: Prefer **fill** when data is scarce; prefer **drop** when missingness is small.

Removing Duplicates

```
print("Duplicates:", df.duplicated().sum())  
  
df = df.drop_duplicates()
```

Removing Duplicates

```
print("Duplicates:", df.duplicated().sum())  
  
df = df.drop_duplicates()
```

Why it matters: Duplicate rows can bias training and inflate evaluation scores.

Feature Scaling

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

# Scale numeric feature columns (example)
df[["Hours", "Attendance"]] = scaler.fit_transform(df[["Hours", "Attendance"]])
```

Feature Scaling

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

# Scale numeric feature columns (example)
df[["Hours", "Attendance"]] = scaler.fit_transform(df[["Hours", "Attendance"]])
```

Why important? **Distance-based models** (KNN, SVM, K-means) are sensitive to feature scale. Without scaling, high-range features **dominate** the model.

Encoding Categorical Variables

```
import pandas as pd

# Convert categories to numeric columns (one-hot encoding)
df_encoded = pd.get_dummies(df, columns=["Gender"], drop_first=True)
print(df_encoded.head())
```

Encoding Categorical Variables

```
import pandas as pd

# Convert categories to numeric columns (one-hot encoding)
df_encoded = pd.get_dummies(df, columns=["Gender"], drop_first=True)
print(df_encoded.head())
```

Output (example):

	Age	...	Gender_Male
0	22	...	1
1	24	...	0

Why: Most ML models require **numerical inputs** (not raw text categories).

End-to-End Case Study (We will do in the lap part)

Student performance dataset (EDA → preprocessing → features)

```
import pandas as pd
import seaborn as sns

df = pd.read_csv("students.csv")
df = df.dropna() # basic cleaning

# quick EDA
sns.heatmap(df.corr(numeric_only=True), annot=True)

# features + target
X = df[["Hours", "Attendance"]].to_numpy()
y = df["Marks"].to_numpy()
```

End-to-End Case Study (We will do in the lap part)

Student performance dataset (EDA → preprocessing → features)

```
import pandas as pd
import seaborn as sns

df = pd.read_csv("students.csv")
df = df.dropna() # basic cleaning

# quick EDA
sns.heatmap(df.corr(numeric_only=True), annot=True)

# features + target
X = df[["Hours", "Attendance"]].to_numpy()
y = df["Marks"].to_numpy()
```

Result: clean, explored, and ready-to-train dataset.

Key Takeaways — EDA & Preprocessing

- **Visualize** first to find patterns, relationships, and problems early.
- Use **scatter**, **histograms**, and **heatmaps** as your core EDA toolkit.
- Good **preprocessing** (missing values, duplicates, scaling, encoding) improves model performance.
- **Clean** + **scaled** + **encoded** data is the foundation of successful AI.

Key Takeaways — EDA & Preprocessing

- **Visualize** first to find patterns, relationships, and problems early.
- Use **scatter**, **histograms**, and **heatmaps** as your core EDA toolkit.
- Good **preprocessing** (missing values, duplicates, scaling, encoding) improves model performance.
- **Clean** + **scaled** + **encoded** data is the foundation of successful AI.

Visualize → Clean → Train

Supervised Learning Fundamentals

Machine Learning Problem Types

Most supervised learning problems fall into two main categories:

Problem type	Output type
Regression	continuous numerical value
Classification	discrete category / label

Machine Learning Problem Types

Most supervised learning problems fall into two main categories:

Problem type	Output type
Regression	continuous numerical value
Classification	discrete category / label

Key idea: Choose based on the **target variable** y .

What is Regression?

Regression models predict **continuous** numerical values.

What is Regression?

Regression models predict **continuous** numerical values.

Examples:

- House **price** prediction

What is Regression?

Regression models predict **continuous** numerical values.

Examples:

- House **price** prediction
- Student **marks/GPA** forecasting

What is Regression?

Regression models predict **continuous** numerical values.

Examples:

- House **price** prediction
- Student **marks/GPA** forecasting
- **temperature** or **stock price** prediction

What is Regression?

Regression models predict **continuous** numerical values.

Examples:

- House **price** prediction
- Student **marks/GPA** forecasting
- **temperature** or **stock price** prediction
- **sales revenue** estimation

What is Regression?

Regression models predict **continuous** numerical values.

Examples:

- House **price** prediction
- Student **marks/GPA** forecasting
- **temperature** or **stock price** prediction
- **sales revenue** estimation

What is Regression?

Regression models predict **continuous** numerical values.

Examples:

- House **price** prediction
- Student **marks/GPA** forecasting
- **temperature** or **stock price** prediction
- **sales revenue** estimation

Example equation:

$$\text{Marks} = f(\text{Hours Studied}, \text{Attendance})$$

What is Regression?

Regression models predict **continuous** numerical values.

Examples:

- House **price** prediction
- Student **marks/GPA** forecasting
- **temperature** or **stock price** prediction
- **sales revenue** estimation

Example equation:

$$\text{Marks} = f(\text{Hours Studied}, \text{Attendance})$$

Output: any real number (e.g., 72.5).

What is Classification?

Classification models predict **discrete** categories (labels).

What is Classification?

Classification models predict **discrete** categories (labels).

Examples:

- Email: **Spam** vs Not Spam

What is Classification?

Classification models predict **discrete** categories (labels).

Examples:

- Email: **Spam** vs Not Spam
- Transactions: **Fraud** vs Genuine

What is Classification?

Classification models predict **discrete** categories (labels).

Examples:

- Email: **Spam** vs Not Spam
- Transactions: **Fraud** vs Genuine
- Medical: **Disease** vs Healthy

What is Classification?

Classification models predict **discrete** categories (labels).

Examples:

- Email: **Spam** vs Not Spam
- Transactions: **Fraud** vs Genuine
- Medical: **Disease** vs Healthy
- Images: **Cat** vs Dog

What is Classification?

Classification models predict **discrete** categories (labels).

Examples:

- Email: **Spam** vs Not Spam
- Transactions: **Fraud** vs Genuine
- Medical: **Disease** vs Healthy
- Images: **Cat** vs Dog

What is Classification?

Classification models predict **discrete** categories (labels).

Examples:

- Email: **Spam** vs Not Spam
- Transactions: **Fraud** vs Genuine
- Medical: **Disease** vs Healthy
- Images: **Cat** vs Dog

Output: one class from a predefined set (e.g., {spam, not spam}).

Regression vs Classification

Aspect	Regression	Classification
Output	continuous (e.g., 87.4)	categorical (e.g., Pass/Fail)
Examples	Price, Marks, Temperature	Spam, Disease, Customer churn
Metrics	MAE, RMSE, R^2	Accuracy, Precision, Recall, F1, AUC

Regression vs Classification

Aspect	Regression	Classification
Output	continuous (e.g., 87.4)	categorical (e.g., Pass/Fail)
Examples	Price, Marks, Temperature	Spam, Disease, Customer churn
Metrics	MAE, RMSE, R^2	Accuracy, Precision, Recall, F1, AUC

Tip: Identify the **problem type** early — it determines model choice and evaluation.

A Typical Machine Learning Dataset

Example: Student performance data

Hours	Attendance	Marks (target)
2	70	55
4	80	65
6	85	78
8	95	90

A Typical Machine Learning Dataset

Example: Student performance data

Hours	Attendance	Marks (target)
2	70	55
4	80	65
6	85	78
8	95	90

Features X : Hours, Attendance

Target y : Marks

The Machine Learning Pipeline

Data collection → Cleaning & EDA → Feature engineering



Train/test split → Model training → Evaluation → Prediction

This pipeline applies to **regression**, **classification**, and **deep learning**.

Linear Regression Intuition

The simplest and most interpretable **regression** model:

$$\hat{y} = w_1x_1 + w_2x_2 + \cdots + b$$

Linear Regression Intuition

The simplest and most interpretable **regression** model:

$$\hat{y} = w_1x_1 + w_2x_2 + \cdots + b$$

Meaning of terms:

- **w**: **weights** (importance of each feature)

Linear Regression Intuition

The simplest and most interpretable **regression** model:

$$\hat{y} = w_1x_1 + w_2x_2 + \cdots + b$$

Meaning of terms:

- **w**: **weights** (importance of each feature)
- **b**: **bias** (offset)

Linear Regression Intuition

The simplest and most interpretable **regression** model:

$$\hat{y} = w_1x_1 + w_2x_2 + \cdots + b$$

Meaning of terms:

- **w**: **weights** (importance of each feature)
- **b**: **bias** (offset)
- **$\hat{y}$** : **prediction**

Linear Regression Intuition

The simplest and most interpretable **regression** model:

$$\hat{y} = w_1x_1 + w_2x_2 + \cdots + b$$

Meaning of terms:

- **w**: **weights** (importance of each feature)
- **b**: **bias** (offset)
- **$\hat{y}$** : **prediction**

Linear Regression Intuition

The simplest and most interpretable **regression** model:

$$\hat{y} = w_1x_1 + w_2x_2 + \cdots + b$$

Meaning of terms:

- **w**: **weights** (importance of each feature)
- **b**: **bias** (offset)
- **$\hat{y}$** : **prediction**

This should look familiar from the NumPy linear model example.

Minimal Scikit-Learn Example (Regression)

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

predictions = model.predict(X_test)
```


Minimal Scikit-Learn Example (Regression)

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

predictions = model.predict(X_test)
```

Output (example):

Model trained successfully.

Predictions: [62.3 78.1 91.5 ...]

Key idea: same `fit` → `predict` workflow appears in most ML libraries.

Linear regression (no non-linearity):

$$\hat{y} = wx + b$$

From Regression to Neural Networks

Linear regression (no non-linearity):

$$\hat{y} = wx + b$$

Single neuron (adds **activation**):

$$\hat{y} = \text{Activation}(wx + b)$$

From Regression to Neural Networks

Linear regression (no non-linearity):

$$\hat{y} = wx + b$$

Single neuron (adds **activation**):

$$\hat{y} = \text{Activation}(wx + b)$$

Stacking many neurons + **non-linear activations** $\Rightarrow$ powerful deep learning models.

Overfitting vs Underfitting

- **Underfitting**: model too simple $\Rightarrow$ misses patterns (high **bias**).

Overfitting vs Underfitting

- **Underfitting**: model too simple $\Rightarrow$ misses patterns (high **bias**).
- **Overfitting**: model too complex $\Rightarrow$ memorizes training data (high **variance**).

Overfitting vs Underfitting

- **Underfitting**: model too simple $\Rightarrow$ misses patterns (high **bias**).
- **Overfitting**: model too complex $\Rightarrow$ memorizes training data (high **variance**).

Overfitting vs Underfitting

- **Underfitting**: model too simple $\Rightarrow$ misses patterns (high **bias**).
- **Overfitting**: model too complex $\Rightarrow$ memorizes training data (high **variance**).

Goal: generalize well to **unseen** data.

Standard split (rule of thumb):

- **Training:** 70–80%

learn parameters (w, b)

Train, Validation & Test Sets

Standard split (rule of thumb):

- **Training:** 70–80%
- **Validation:** 10–15%

learn parameters (w, b)

tune hyperparameters

Standard split (rule of thumb):

- **Training:** 70–80%
- **Validation:** 10–15%
- **Test:** 10–20%

learn parameters (w, b)

tune hyperparameters

final unbiased evaluation

Standard split (rule of thumb):

- **Training:** 70–80%
- **Validation:** 10–15%
- **Test:** 10–20%

learn parameters (w, b)

tune hyperparameters

final unbiased evaluation

Train, Validation & Test Sets

Standard split (rule of thumb):

- **Training:** 70–80% learn parameters (w, b)
- **Validation:** 10–15% tune hyperparameters
- **Test:** 10–20% final unbiased evaluation

Keeping them separate prevents **data leakage** and gives a reliable estimate of performance.

Key Takeaways — Supervised Learning Fundamentals

- First decide: **regression** or **classification**.

Regression & **Classification** = foundation of supervised learning

Key Takeaways — Supervised Learning Fundamentals

- First decide: **regression** or **classification**.
- Data = **features** X + **target** y .

Regression & **Classification** = foundation of supervised learning

Key Takeaways — Supervised Learning Fundamentals

- First decide: **regression** or **classification**.
- Data = **features** X + **target** y .
- **Linear regression** is the starting point for understanding neural networks.

Regression & Classification = foundation of supervised learning

Key Takeaways — Supervised Learning Fundamentals

- First decide: **regression** or **classification**.
- Data = **features** X + **target** y .
- **Linear regression** is the starting point for understanding neural networks.
- Always evaluate on **unseen** data (train/val/test split).

Regression & Classification = foundation of supervised learning

Key Takeaways — Supervised Learning Fundamentals

- First decide: **regression** or **classification**.
- Data = **features** X + **target** y .
- **Linear regression** is the starting point for understanding neural networks.
- Always evaluate on **unseen** data (train/val/test split).
- Avoid **overfitting**; aim for **generalization**.

Regression & Classification = foundation of supervised learning

Model Evaluation

Why Model Evaluation Matters

After training a model, the critical question is:

How good is this model on **unseen data?**

Why Model Evaluation Matters

After training a model, the critical question is:

How good is this model on **unseen data?**

A good model must:

- Perform well on **unseen** examples (not memorization)

Why Model Evaluation Matters

After training a model, the critical question is:

How good is this model on **unseen data?**

A good model must:

- Perform well on **unseen** examples (not memorization)
- **Generalize** beyond the training set

Why Model Evaluation Matters

After training a model, the critical question is:

How good is this model on **unseen data?**

A good model must:

- Perform well on **unseen** examples (not memorization)
- **Generalize** beyond the training set
- Produce **reliable** and **trustworthy** predictions

Why Model Evaluation Matters

After training a model, the critical question is:

How good is this model on **unseen data?**

A good model must:

- Perform well on **unseen** examples (not memorization)
- **Generalize** beyond the training set
- Produce **reliable** and **trustworthy** predictions

Why Model Evaluation Matters

After training a model, the critical question is:

How good is this model on **unseen data?**

A good model must:

- Perform well on **unseen** examples (not memorization)
- **Generalize** beyond the training set
- Produce **reliable** and **trustworthy** predictions

Different tasks need different **metrics**.

Evaluation Depends on the Problem

Task	Recommended metrics
Regression	MAE, RMSE, R^2
Classification (balanced)	Accuracy, F1-score
Classification (imbalanced)	Precision, Recall, F1, ROC-AUC

Evaluation Depends on the Problem

Task	Recommended metrics
Regression	MAE, RMSE, R^2
Classification (balanced)	Accuracy, F1-score
Classification (imbalanced)	Precision, Recall, F1, ROC-AUC

No single best metric: choose based on **real-world cost** of errors.

Regression Evaluation Example

Suppose we have **actual** vs **predicted** values:

Actual	Predicted
80	78
70	72
90	88
60	63

Regression Evaluation Example

Suppose we have **actual** vs **predicted** values:

Actual	Predicted
80	78
70	72
90	88
60	63

Evaluation measures **error**: how close predictions are to actual values.

Regression Metrics in Scikit-Learn

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

y_true = np.array([80, 70, 90, 60])
y_pred = np.array([78, 72, 88, 63])

print("MAE:", mean_absolute_error(y_true, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_true, y_pred)))
print("R2 Score:", r2_score(y_true, y_pred))
```

Output:

MAE: 2.25

RMSE: 2.5

R² Score: 0.96875

Lower MAE/RMSE and higher R^2 indicate better performance.

Classification models predict **categories**, so we summarize results using a **confusion matrix**.

Classification models predict **categories**, so we summarize results using a **confusion matrix**.

Goal: understand **what goes right** and **what goes wrong** (not just one number).

Confusion Matrix

	Pred +	Pred -
Actual +	TP	FN
Actual -	FP	TN

Confusion Matrix

	Pred +	Pred -
Actual +	TP	FN
Actual -	FP	TN

- **TP** (true positive): predicted positive and actually positive
- **TN** (true negative): predicted negative and actually negative
- **FP** (false positive): predicted positive but actually negative
- **FN** (false negative): predicted negative but actually positive

Classification Metrics in Scikit-Learn

```
from sklearn.metrics import (  
    accuracy_score, precision_score, recall_score, f1_score  
)  
  
y_true = [1, 0, 1, 1, 0, 1]  
y_pred = [1, 0, 0, 1, 0, 1]  
  
print("Accuracy:", accuracy_score(y_true, y_pred))  
print("Precision:", precision_score(y_true, y_pred))  
print("Recall:", recall_score(y_true, y_pred))  
print("F1 Score:", f1_score(y_true, y_pred))
```

Classification Metrics in Scikit-Learn

```
from sklearn.metrics import (  
    accuracy_score, precision_score, recall_score, f1_score  
)  
  
y_true = [1, 0, 1, 1, 0, 1]  
y_pred = [1, 0, 0, 1, 0, 1]  
  
print("Accuracy:", accuracy_score(y_true, y_pred))  
print("Precision:", precision_score(y_true, y_pred))  
print("Recall:", recall_score(y_true, y_pred))  
print("F1 Score:", f1_score(y_true, y_pred))
```

Output:

Accuracy: 0.8333

Precision: 0.75

Recall: 1.0

F1 Score: 0.8571

Why Accuracy Can Be Misleading

Consider a **fraud detection** dataset:

- 990 **legitimate** transactions
- 10 **fraudulent** transactions

Why Accuracy Can Be Misleading

Consider a **fraud detection** dataset:

- 990 **legitimate** transactions
- 10 **fraudulent** transactions

A model that predicts **everything** as **legitimate** gets **99% accuracy** but detects **zero frauds**.

Why Accuracy Can Be Misleading

Consider a **fraud detection** dataset:

- 990 **legitimate** transactions
- 10 **fraudulent** transactions

A model that predicts **everything** as **legitimate** gets **99% accuracy** but detects **zero frauds**.

For **imbalanced data**, prefer **precision**, **recall**, **F1**, and **ROC-AUC**.

ROC-AUC

- **ROC curve**: plots **TPR** vs **FPR** across thresholds
- **AUC**: single number summarizing ranking quality

ROC-AUC

- **ROC curve**: plots **TPR** vs **FPR** across thresholds
- **AUC**: single number summarizing ranking quality

Interpretation:

AUC	Quality
0.5	Random guessing
0.7–0.8	Acceptable
0.8–0.9	Good
0.9+	Excellent

Key Takeaways — Model Evaluation

- Evaluate on **unseen test data**.
- Pick **metrics** based on the task and real-world costs.
- Regression: common metrics include **MAE/RMSE**.
- Classification: common metrics include **F1** and **ROC-AUC** (for probabilistic models).
- Good evaluation prevents deploying **unreliable** models.

Key Takeaways — Model Evaluation

- Evaluate on **unseen test data**.
- Pick **metrics** based on the task and real-world costs.
- Regression: common metrics include **MAE/RMSE**.
- Classification: common metrics include **F1** and **ROC-AUC** (for probabilistic models).
- Good evaluation prevents deploying **unreliable** models.

Train → Evaluate → Deploy

TensorFlow

Why TensorFlow?

TensorFlow is a popular open-source framework for **machine learning** and **deep learning**.

- **Speed** with **GPU/TPU** support

Why TensorFlow?

TensorFlow is a popular open-source framework for **machine learning** and **deep learning**.

- **Speed** with **GPU/TPU** support
- **Keras API**: high-level and beginner-friendly

Why TensorFlow?

TensorFlow is a popular open-source framework for **machine learning** and **deep learning**.

- **Speed** with **GPU/TPU** support
- **Keras API**: high-level and beginner-friendly
- **Automatic differentiation** $\Rightarrow$ backpropagation

Why TensorFlow?

TensorFlow is a popular open-source framework for **machine learning** and **deep learning**.

- **Speed** with **GPU/TPU** support
- **Keras API**: high-level and beginner-friendly
- **Automatic differentiation** $\Rightarrow$ backpropagation
- **Production** tools for deployment

Why TensorFlow?

TensorFlow is a popular open-source framework for **machine learning** and **deep learning**.

- **Speed** with **GPU/TPU** support
- **Keras API**: high-level and beginner-friendly
- **Automatic differentiation** $\Rightarrow$ backpropagation
- **Production** tools for deployment

Why TensorFlow?

TensorFlow is a popular open-source framework for **machine learning** and **deep learning**.

- **Speed** with **GPU/TPU** support
- **Keras API**: high-level and beginner-friendly
- **Automatic differentiation** $\Rightarrow$ backpropagation
- **Production** tools for deployment

Learning path: NumPy $\rightarrow$ Pandas $\rightarrow$ TensorFlow $\rightarrow$ Deep learning

From Arrays to Tensors

TensorFlow's core data structure is the **tensor** — a generalization of NumPy arrays.

From Arrays to Tensors

TensorFlow's core data structure is the **tensor** — a generalization of NumPy arrays.

- **0D**: scalar (single number)
- **1D**: vector
- **2D**: matrix
- **nD**: tensor

From Arrays to Tensors

TensorFlow's core data structure is the **tensor** — a generalization of NumPy arrays.

- **0D**: scalar (single number)
- **1D**: vector
- **2D**: matrix
- **nD**: tensor

If you understand **shape** and **axes** in NumPy, you already understand most tensor concepts.

Creating Tensors

```
import tensorflow as tf

a = tf.constant([1, 2, 3])
b = tf.constant([4, 5, 6])

print(a)
print(a + b)  # elementwise addition
```

Creating Tensors

```
import tensorflow as tf

a = tf.constant([1, 2, 3])
b = tf.constant([4, 5, 6])

print(a)
print(a + b)  # elementwise addition
```

Output:

```
tf.Tensor([1 2 3], shape=(3,), dtype=int32)
tf.Tensor([5 7 9], shape=(3,), dtype=int32)
```

Notice: TensorFlow shows **values**, **shape**, and **dtype**.

Tensor Shapes

```
import tensorflow as tf

x = tf.constant([[1, 2, 3],
                 [4, 5, 6]])

print(x.shape)
print(x.dtype)
```


Tensor Shapes

```
import tensorflow as tf

x = tf.constant([[1, 2, 3],
                 [4, 5, 6]])

print(x.shape)
print(x.dtype)
```

Output:

```
(2, 3)
<dtype: 'int32'>
```

Why it matters: neural networks expect specific **input dimensions**.

Integration with NumPy & Pandas

```
import numpy as np
import pandas as pd
import tensorflow as tf

# From NumPy
arr = np.array([1, 2, 3])
t = tf.constant(arr)

# From Pandas DataFrame
# X = df[["feature1", "feature2"]].to_numpy()
# X_tf = tf.constant(X)
```

Integration with NumPy & Pandas

```
import numpy as np
import pandas as pd
import tensorflow as tf

# From NumPy
arr = np.array([1, 2, 3])
t = tf.constant(arr)

# From Pandas DataFrame
# X = df[["feature1", "feature2"]].to_numpy()
# X_tf = tf.constant(X)
```

Idea: Pandas → NumPy → TensorFlow tensors.

Key Takeaways — TensorFlow & Neural Networks

- **Tensors** are the core data structure (like NumPy arrays).

Key Takeaways — TensorFlow & Neural Networks

- **Tensors** are the core data structure (like NumPy arrays).
- **Keras** makes building neural networks simple and readable.

Key Takeaways — TensorFlow & Neural Networks

- **Tensors** are the core data structure (like NumPy arrays).
- **Keras** makes building neural networks simple and readable.
- Neural networks = **layers** + **non-linear activations**.

Key Takeaways — TensorFlow & Neural Networks

- **Tensors** are the core data structure (like NumPy arrays).
- **Keras** makes building neural networks simple and readable.
- Neural networks = **layers** + **non-linear activations**.
- Training loop: forward pass → **loss** → **backprop**.

Key Takeaways — TensorFlow & Neural Networks

- **Tensors** are the core data structure (like NumPy arrays).
- **Keras** makes building neural networks simple and readable.
- Neural networks = **layers** + **non-linear activations**.
- Training loop: forward pass → **loss** → **backprop**.
- Good **preprocessing** is essential for good performance.

Key Takeaways — TensorFlow & Neural Networks

- **Tensors** are the core data structure (like NumPy arrays).
- **Keras** makes building neural networks simple and readable.
- Neural networks = **layers** + **non-linear activations**.
- Training loop: forward pass → **loss** → **backprop**.
- Good **preprocessing** is essential for good performance.

Key Takeaways — TensorFlow & Neural Networks

- **Tensors** are the core data structure (like NumPy arrays).
- **Keras** makes building neural networks simple and readable.
- Neural networks = **layers** + **non-linear activations**.
- Training loop: forward pass → **loss** → **backprop**.
- Good **preprocessing** is essential for good performance.

NumPy + Pandas + TensorFlow = foundation for modern AI